

LAB2 – INF1316 – Sistemas Operacionais – 2026.1

Criação de Processos Irmãos

1. Identificação

Nome: Caio Faria Brito Martins Chaves

Matrícula: 2410162

Nome: Lucas Hufnagel Gromann de Araujo Goes

Matrícula: 2410845

2. Objetivo

O objetivo deste laboratório é implementar um programa em linguagem C que demonstre a criação de múltiplos processos utilizando a chamada de sistema `fork()` em sistemas operacionais do tipo Unix.

O programa deve criar três processos filhos (irmãos), todos originados diretamente do processo pai. Cada um desses processos executa um conjunto de operações sobre uma variável inteira `n`, inicialmente igual a 1, realizando incrementos distintos e exibindo informações como o PID do processo e o valor final da variável.

Além disso, o programa permite observar o comportamento da execução concorrente dos processos e analisar o isolamento de memória entre eles.

3. Estrutura do Programa

O programa foi desenvolvido em linguagem C utilizando as seguintes bibliotecas:

`stdio.h`

`unistd.h`

`sys/wait.h`

`stdlib.h`

A implementação está concentrada na função `main`, onde:

- A variável inteira `n` é inicializada com valor 1;

- São realizadas três chamadas do `fork()`, responsáveis pela criação dos processos;
- Cada processo executa um for de 1000 iterações realizando operações distintas sobre `n`;
- Cada processo imprime seu PID utilizando a função `getpid()` e o valor atualizado da variável;
- A função `exit()` é utilizada para finalizar os processos após sua execução.

4. Solução

O programa inicia declarando a variável `n` com valor inicial igual a 1. Em seguida, são realizadas três chamadas consecutivas de `fork()`, armazenadas nas variáveis `filho1`, `filho2` e `filho3`.

Após a criação dos processos, cada bloco condicional verifica o valor retornado por `fork()`. Quando o valor é diferente de zero, o código correspondente é executado. Assim, determinados processos entram em cada um dos blocos e executam suas respectivas tarefas.

Os comportamentos implementados são:

- Um processo executa um loop de 1000 iterações somando 1 à variável `n`;
- Outro processo executa um loop de 1000 iterações somando 10 à variável `n`;
- Outro processo executa um loop de 1000 iterações somando 100 à variável `n`.

Durante a execução dos loops, cada processo imprime mensagens contendo seu PID e o valor atualizado de `n`. Ao final, cada processo imprime o valor total obtido e encerra sua execução com `exit()`.

Como os processos são executados concorrentemente pelo sistema operacional, as saídas aparecem intercaladas no terminal, evidenciando paralelismo.

5. Observações e Conclusões

Os valores impressos pelos processos são diferentes. Isso ocorre porque, após cada chamada de `fork()`, o processo filho recebe uma cópia independente do espaço

de memória do processo pai. Assim, cada processo possui sua própria variável n , sem interferência dos demais.

Dessa forma, mesmo que todos os processos tenham sido criados a partir do mesmo valor inicial ($n = 1$), cada um realiza suas operações de forma isolada.

Os valores finais esperados são:

- Processo que soma 1: $n = 1001$
- Processo que soma 10: $n = 10001$
- Processo que soma 100: $n = 100001$

Os resultados observados são consistentes com o esperado, considerando que cada processo executa 1000 iterações com incrementos fixos.